

NeuroBrush: Real-Time Artistic Style Transfer System

Course: AIT102 - Python and TensorFlow Programming

December 16, 2025

1 Group Members

No.	Name	Student ID
1	Min Yiduo (Leader)	DSC2409026
2	Cui Zeyu	DSC2409006
3	Deng Kailong	DSC2409007
4	Dong Yanzhe	DSC2409008
5	Lin Jiacheng	DSC2409018

2 Task Distribution

- **Min Yiduo:** Model Architecture Design & Research
- **Deng Kailong:** Dataset Preparation (COCO 2017) & Preprocessing Pipeline
- **Dong Yanzhe:** Backend Implementation (TensorFlow Training Loop)
- **Cui Zeyu:** Frontend Development (HTML/Flask Web Interface)
- **Lin Jiacheng:** Testing, Optimization & Assignment Report Writing

3 Model Description

3.1 Overview

Our project aims to implement a Fast Style Transfer system capable of transforming user-uploaded photos into artistic styles (e.g., Van Gogh, Picasso) in real-time. Unlike the slow optimization-based method (Gatys et al. [1]), we will train a feed-forward Image Transformation Network using the architecture proposed by Johnson et al. [2]

3.2 Development Phases

The system consists of two distinct phases: Training Phase and Inference Phase, as shown in fig. 1.

3.2.1 Training Phase (Development)

In this phase, we train the Transformation Network to learn a specific style.

- **Dataset:** We use the COCO 2017 Dataset (approx. 80,000 images) to train the network on content recognition.
- **Style Image:** A single artistic image (e.g., *Starry Night*) serves as the target style.
- **Loss Network:** We use a pre-trained VGG-19 network to calculate:
 - *Content Loss*: Difference in feature maps between generated image and original content image.
 - *Style Loss*: Difference in Gram matrices between generated image and style image.
- **Optimizer:** We use Adam optimizer to update the weights of the Transformation Network to minimize the total loss.

3.2.2 Inference Phase (Production)

Once trained, the VGG-19 Loss Network is discarded.

- **Input:** User provides a content image (via HTML frontend).
- **Processing:** The trained Transformation Network acts as a filter, processing the image in a single forward pass.
- **Output:** A stylized image is generated in milliseconds.

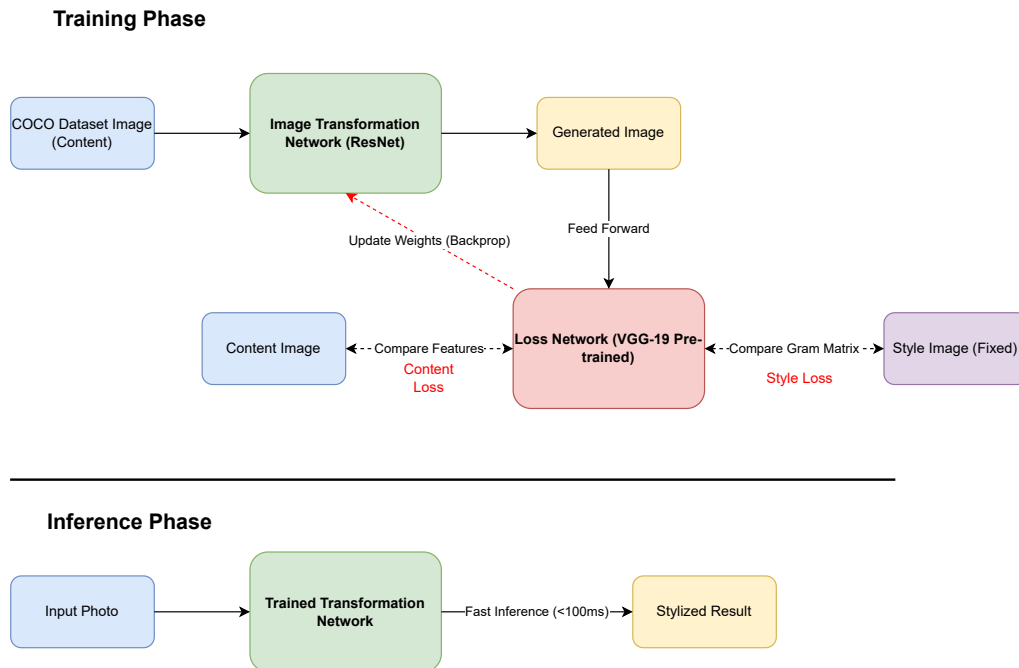


Figure 1: Two Phases of Project Development

4 System Architecture

The final application will features a web-based interface, as shown in fig. 2:

- **Frontend:** An HTML5/CSS3 web page allowing users to upload images and select styles.
- **Backend:** A Python Flask server hosting the TensorFlow model.
- **Workflow:**
 1. User uploads image on Web UI.
 2. Image is sent to Python backend.
 3. TensorFlow model performs inference.
 4. Stylized image is returned and displayed on the webpage.

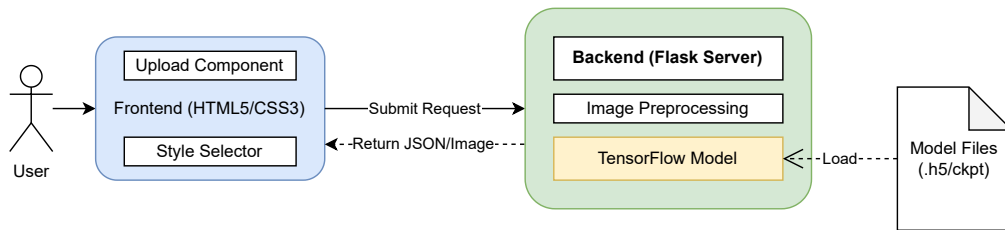


Figure 2: System Architecture

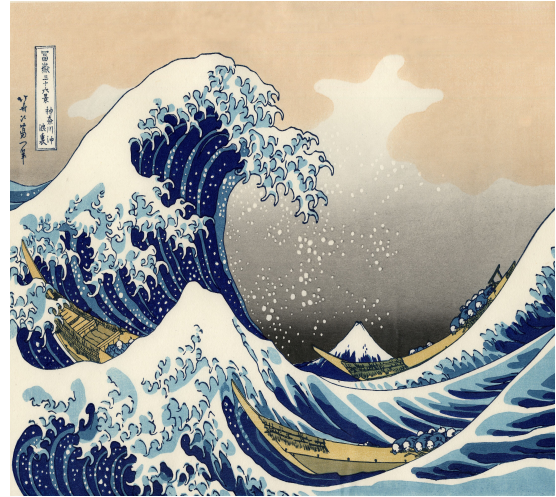
Note on Current Progress (Submission)

This folder contains the Phase 1 (MVP) of the project.

- We have implemented the complete inference pipeline and image processing utilities.
- To demonstrate functionality immediately, the current code utilizes a pre-trained TF-Hub model as a placeholder.
- Phase 2 (In Progress): We are currently setting up the training pipeline on the COCO dataset to replace the placeholder with our own custom-trained weights as described above.
- Demonstration: Figure 3 shows the input images used for our MVP, and Figure 4 displays the generated stylized result.



Content Image



Style Image

Figure 3: Input Images for Style Transfer



Figure 4: Stylized Result (MVP Phase)

References

- [1] Leon A Gatys, Alexander S Ecker, and Matthias Bethge. A neural algorithm of artistic style. *arXiv preprint arXiv:1508.06576*, 2015.
- [2] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European conference on computer vision*, pages 694–711. Springer, 2016.